LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 7 – Abstract dan Interface



NAMA: Anindya Sukma Dwiyanda

NIM: 103132400006

KELAS: S1SD 05 01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Abstract class merupakan kelas yang tidak dapat dibuat menjadi objek secara langsung dan digunakan sebagai kelas induk yang dapat diwariskan ke kelas turunannya. Sementara itu, interface digunakan untuk mendefinisikan method yang harus diimplementasikan oleh kelas lain. Dalam Java, interface juga mendukung multiple inheritance, serta memiliki default method dan static method. Kedua konsep ini membantu membuat program lebih terstruktur, fleksibel, dan mudah dikembangkan.

B. Guided

1. Guided 1 – Implementasi Abstract Class dan Inheritance

```
package com.indy.guided1;


public abstract class Employee {
    private String name;


    public Employee(String name){
        this.name = name;
    }


    public String getName() {
        return name;
    }


    public abstract double earnings();
}
```

```
package com.indy.guided1;

public class SalariedEmployee extends Employee{
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
```

```
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

```
package com.indy.guided1;

public class Main {
    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("Budi", 5000000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" + salaried.earnings());
    }
}
```

Output :

```
Nama: Budi
Gaji: Rp5000000.0
```

2. Guided 2 – Implementasi Interface pada Java

```
package com.indy.guided2;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
```

```
        return earnings();
    }
}
```

```
package com.indy.guided2;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

```
package com.indy.guided2;

public interface Payable {
    double getPaymentAmount();
}
```

```
package com.indy.guided2;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
```

```
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

```
package com.indy.guided2;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];

        items[0] = new Invoice("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable item : items) {
            System.out.println("jumlah yang perlu dibayar: Rp" +
item.getPaymentAmount());
        }
    }
}
```

Output :

```
jumlah yang perlu dibayar: Rp3000000.0
jumlah yang perlu dibayar: Rp5000000.0
```

3. Guided 3 – Implementasi Default Method dan Static Method pada Interface

```
package com.indy.guided3;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
```

```
    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

```
package com.indy.guided3;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

```
package com.indy.guided3;

public interface Payable {
    double getPaymentAmount();

    default void printReceipt() {
        System.out.println("Struk Resmi - Total: Rp" + getPaymentAmount());
    }

    static void logTransaction(String message) {
```

```
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

```
package com.indy.guided3;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

```
package com.indy.guided3;

public class Main {
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran diinisialisasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DOR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable item : items) {
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");
    }

}
```

Output :

```
[AUDIT LOG] Sistem pembayaran diinisialisasi.
Struk Resmi - Total: Rp4000000.0
Struk Resmi - Total: Rp5000000.0
[AUDIT LOG] Transaksi selesai diproses.
```

C. Unguided

1. Soal Unguided 1

   Membuat TiketBioskop abstrak dengan atribut dan getter, serta dua subclass TiketReguler dan TiketVIP yang menghitung biaya sesuai rumus masing-masing.

   Jawaban :

```
package com.indy.unguided;


public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
```

```
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

```
package com.indy.unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi,
            int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
```

```
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
    }
}
```

```
package com.indy.unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi,
            double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP) * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp"
                + (getHargaDasar() + biayaFasilitasVIP) + ")";
    }
}
```

Penjelasan :
TiketBioskop tidak dapat diinstansiasi karena abstrak. Constructor subclass memakai super() untuk meneruskan data ke kelas induk. TiketReguler mengalikan harga dasar dengan jumlah kursi, sedangkan TiketVIP menambahkan biaya fasilitas untuk setiap kursi.

2. Soal Unguided 2

   Membuat ItemTransaksi dan PaketSnack serta menghubungkan TiketBioskop dengan interface melalui implements.

   Jawaban :

```
package com.indy.unguided;


public interface ItemTransaksi {
    double getNominalBayar();


    String getNamaTransaksi();


    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi()
                + " | Total: Rp" + getNominalBayar());
    }


    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

```
package com.indy.unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi) {
        this.namaPaket = namaPaket;
```

```
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return namaPaket;
    }

    public int getJumlahPorsi() {
        return jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket
                + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi + ")";
    }
}
```

Penjelasan :
Interface ItemTransaksi menjadi kontrak bersama untuk tiket dan snack yang tidak memiliki hubungan inheritance langsung. TiketBioskop mengarahkan getNominalBayar() ke hitungTotalBayar(), sedangkan PaketSnack menghitung jumlahPorsi × hargaPerPorsi.

3. Soal Unguided 3
   Menambahkan default method cetakRincianItem() dan static method catatLogKasir(String pesan) pada ItemTransaksi.

   Jawaban :

```
package com.indy.unguided;
```

```
public interface ItemTransaksi {
    double getNominalBayar();

    String getNamaTransaksi();

    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi()
                + " | Total: Rp" + getNominalBayar());
    }

    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

Penjelasan :
Default method mencetak nama dan nominal berdasarkan implementasi objek aktual, sedangkan static method mencetak log kasir melalui ItemTransaksi.catatLogKasir(). Tidak diperlukan override pada masing-masing kelas.

4. Soal Unguided 4

Membuat Main, mengisi array ItemTransaksi[] dengan tiga jenis objek, mencetak rincian, dan mengakumulasi grand total.

```
package com.indy.unguided;

public class Main {
    public static void main(String[] args) {
        // TiketBioskop bersifat abstract sehingga tidak dapat diinstansiasi.
        // TiketBioskop tiket = new TiketBioskop("Interstellar", 40000, 2);
```

```
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");

        TiketReguler tiketReguler = new TiketReguler("Interstellar", 40000, 2, 3);
        TiketVIP tiketVIP = new TiketVIP("Dune: Part Two", 60000, 2, 25000);
        PaketSnack paketSnack = new PaketSnack("Combo Popcorn Caramel + 2 Soda",
1, 55000);

        ItemTransaksi[] keranjang = {tiketReguler, tiketVIP, paketSnack};

        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");

        double totalTransaksi = 0;
        for (ItemTransaksi item : keranjang) {
            item.cetakRincianItem();
            totalTransaksi += item.getNominalBayar();
        }

        System.out.println("------------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + totalTransaksi);

System.out.println("============================================================
==============");

        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Output:

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Penjelasan:

Array ItemTransaksi[] menampung objek heterogen. Perulangan memanggil default method dan getNominalBayar() secara polimorfik. Total = 80.000 + 170.000 + 55.000 = Rp305.000. Baris instansiasi abstract class dijadikan komentar sesuai instruksi soal.

D. Kesimpulan

Praktikum Modul 7 menunjukkan bahwa abstract class dapat menjadi kerangka umum bagi subclass yang memiliki atribut dan perilaku terkait, sementara interface memungkinkan objek dari hierarki berbeda diproses melalui kontrak yang sama. Pada guided, konsep tersebut ditunjukkan melalui Employee–SalariedEmployee, Payable–Invoice, Product–Shoe, serta contoh default dan static method. Pada unguided, sistem kasir bioskop menggunakan ItemTransaksi[] untuk memproses tiket dan snack secara polimorfik dengan total pembayaran Rp305.000. Dengan demikian, penggunaan abstraksi dan interface membantu membuat struktur program lebih konsisten, modular, dan mudah dikembangkan.